DREW

# Lecture 35: Linked List

CSCI 260 Computer Systems & Architecture
Fall 2024

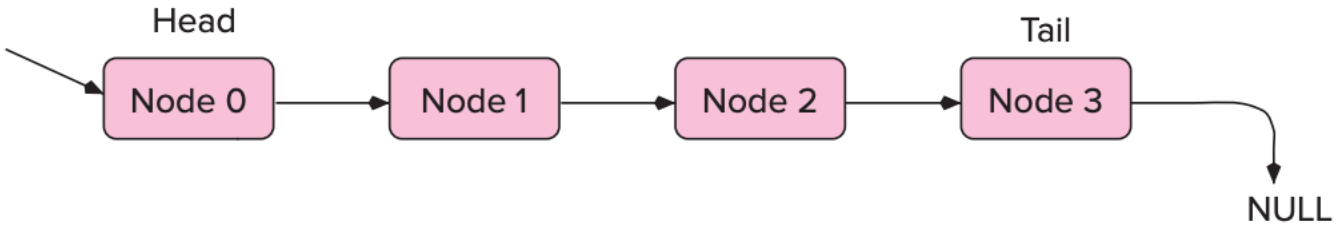
Andy Siyuan Feng

11/25/2024

# Data Structure – Linked List

- A particular widely used data structure is call a *Linked List*.

- A LL is like an array: data store in a sequential form, each entry is a *node*.
  - The first node is referred as the *head* of the LL.
  - Like an array, we use this first node to *identify the entire LL*.
  - The LL can have a second node, referred as the *next* of the *head*. [Linked!! Here *Singly Linked*.]
  - If there's a third, then it is the *next of next of the head*.




  - Until the last node, which has no *next*; thus, its next is NULL.

*Referred as the the *tail* of the LL. [*: NO link back!!]

DREW

# Data Structure – Linked List

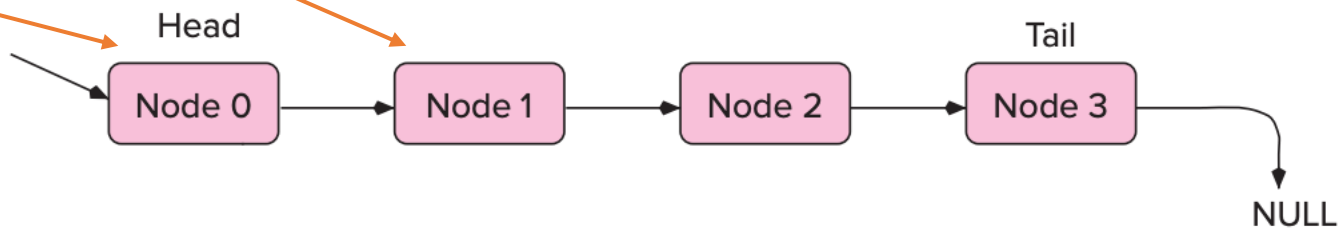
- A *LL node* is implemented with a *struct* in C.

```
typedef struct LL_Node {
        some vars.;
        node* next;
} node;
```

The addr of the next node

- So, use *struct* notion, to get to Node 2, we need to access:

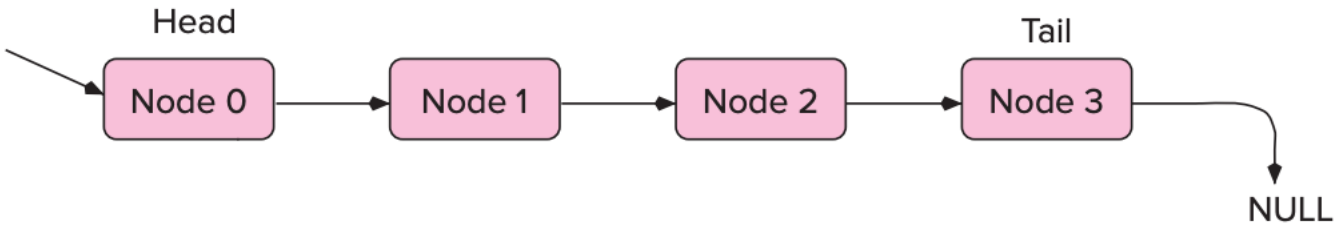
*((head->next)->next)->next*

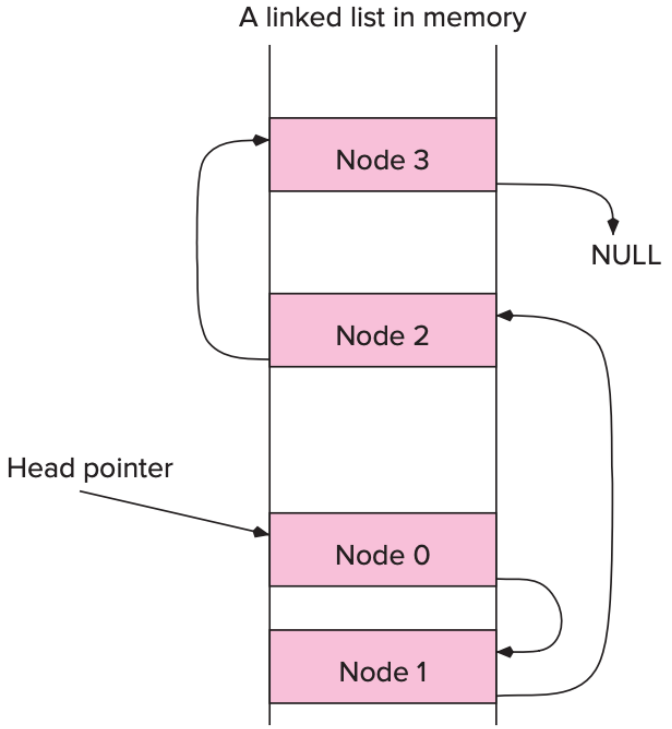



DREW

# Data Structure – Linked List

- Since each instantiation of a new *node** will create a slab in the memory, unlike arrays, data locations in a LL are NOT necessarily adjacent in the memory.




- **Advantage** of this: Create a node when needed;

i.e., dynamic size; can update by adding/deleting nodes.

DREW

# Linked List Functions

- To implement a linked list, we will need helper functions.
  1. To set a node with its attributes: what values to hold; and what is the *next* of this node.
  2. To append a node to LL: If LL is empty, this node is the LL; else go to *tail*, make this the new *tail*.
  3. To check if LL is empty.
  4. To add a node at a location: Insert node between two existing ones.
  5. To delete a node: Delete a node if exists in LL and reconnect the nodes before and after.
  6. To check if node exists in LL.
  7. To print LL.

# Linked List Basic Functions

- To set a node:
  - Pass in the data; set the data to attributes;
  - Set *next* to NULL.

- To check if LL is empty:
  - Pass in the *head** and compare with NULL;
  - Return different codes: 1 for empty, 0 otherwise.

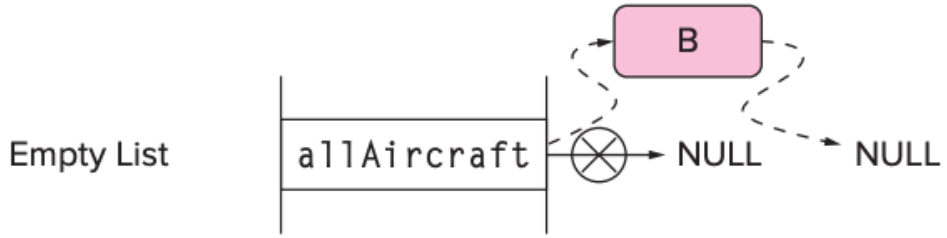
- To print LL:
  - Pass in the head* and check if empty.
  - If not, print data of current node and move to *next*.
  - Until current becomes NULL.

# Linked List Basic Functions

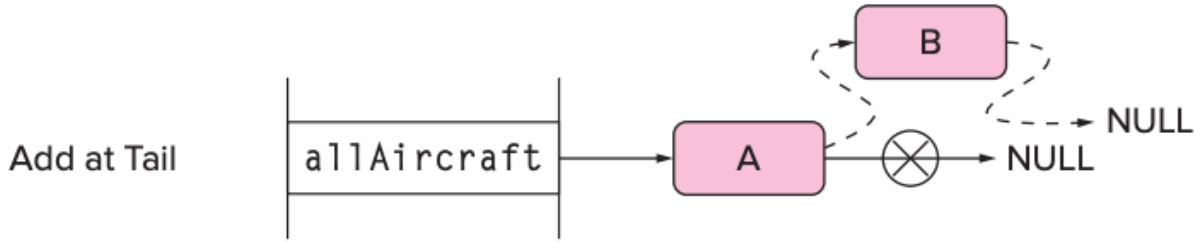
- To check if node exists in LL:
  - Pass in the head* and check if empty. If empty, return 0 as not exists.
  - Compare data of current node and the one to check with.
  - If match, return 1 as exists.
  - If not, move to *next* in the LL.
  - Repeat until current becomes NULL and return 0.

# Linked List Essential Functions

- To append a node to LL: 2 cases to consider within *appendLL()*.
  - **If LL is empty**: simply set the LL as this node [both struct*].




  - **If LL is not empty**: need to get to the tail and make this node the new tail.

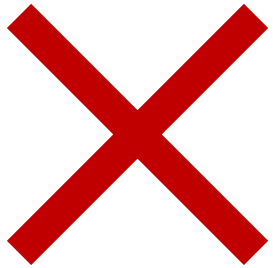


# Linked List Essential Functions

- To append a node to LL:

```
void appendLL(node* head, int val) {
        node* new_node = setNode(val);

        if (isEmpty(head) == 1)
                head = new_node;
        else {
                // iterate to tail and append
        }
}
```



Here, we want to modify head as an LL.

Recall the in-class that a string was passed into a function to be modified.

If we simply pass it as a char*, then the change is not going to be made in the caller function.

Similarly, here!

DREW

# Linked List Essential Functions

- To append a node to LL: [The correct way!]

```
void appendLL(node** head, int val) {
        node* new_node = setNode(val);

        if (isEmpty(*head) == 1)
                *head = new_node;
        else {
                // iterate to tail and append
        }
}
```

# Linked List Essential Functions

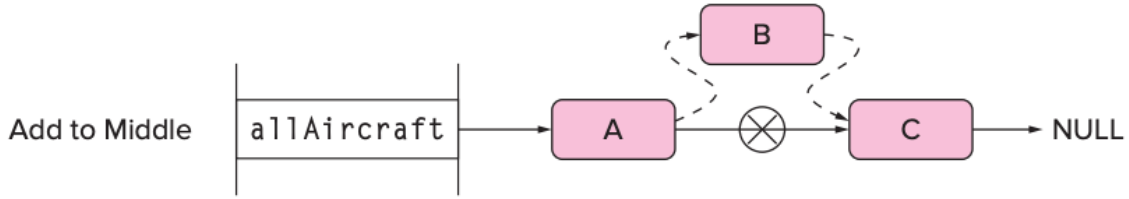
- To append a node to LL: [The correct way!]

```
        … …
        else {
                // iterate to tail and append
                node* curr = *head;
                while (curr->next != NULL)
                        curr = curr->next;
                curr->next = new_node;
        }
}
```

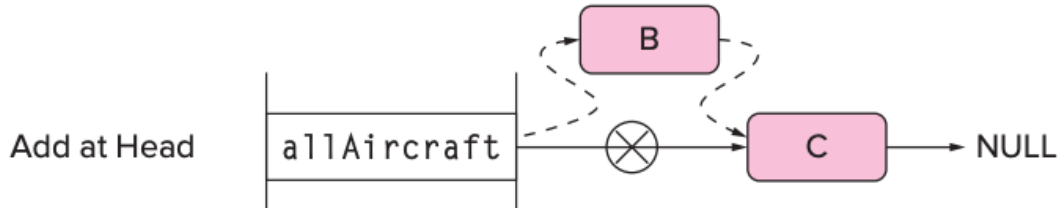
DREW

# Linked List Essential Functions

- To add a node at a location: 5 cases to consider within *addLL()*.
  - If LL is empty: [DONE]
  - **If LL is not empty**: compare some values to determine location.
    - Add to tail. [DONE]
    - **Add to middle**: [Need to modify 2 links, one before location and one after.




    - **Add at head**:




    - ~~**Node already exist**: Besides detecting, nothing to be done.~~

# Linked List Essential Functions

- To add a node to the middle of LL:

```
void addLL(node** head, int val) {
    … …
    // block for add to middle
    node* curr = *head;        node* prev;
    while (curr->val < val) {  // also need to pay attention to tail
        prev = curr;
        curr = curr->next;
    }
    prev->next = new_node;
    new_node->next = curr;
```

# Linked List Essential Functions

- To delete a node based on a value: Similar techniques when dealing with the links.
  1. Find the node being deleted (whether exists?), while keeping track of the previous node.
  2. Reconnect the previous node and the next node.
  3. Lastly, *free()* the node being deleted.

- Since will also want to modify LL, need to pass in a double pointer,

*void deleteLL(node** head, int val) { … }*

# In *main()*

- Lastly, in main:
  - How to pass to say *appendLL(node**, int)*?
  - Say we have a LL *node* myLL*.
  - Need a double pointer, so pass a double pointer; i.e., the address of *myLL*, who's also an address. Thus, *&myLL* is a double pointer.
  - Similarly with *addLL(node**, int), deleteLL(node**, int)*.

# Short Summary

- We just saw how struct can be used for implementing a **linked list**.
- Can also be used for implementing many other data structures; e.g., a **stack**, a **queue**, and etc..

- On the other hand, a linked list can be used as a basis for implementing even more advanced data structures; e.g., **trees**, **heaps**, and etc.